

COPILOTTRAINING
TECHNICAL BRIEFING

Harness Fundamentals

Getting Inside the Agent Loop

Host Copilot's agent loop in your own code, decide what it sees, and keep the repository floor underneath.

Foundations · CopilotTraining Tech Talk · 60 min · @github/copilot-sdk 1.0.26

🤔 THE CORE QUESTION

"How do I get inside the agent loop and change how it behaves?"

Host the same loop as Copilot CLI and decide what the agent sees, uses, and reports.

24 tool calls · zero source reads · three onboarding findings

Developers

Run the agent loop from your own TypeScript and steer each tool call

Platform engineers

Keep repository and policy hooks in force under every host your teams build

Architects

Decide which workflow earns its own host and who reads its report

5 runnable steps

From npm install to a permission handler, each with observed output

2 working apps

Fresh-eyes onboarding check and blind test writer, run against FanHub

19 tests

Tested hook decisions and visibility rules in examples/

📖 TABLE OF CONTENTS

PART 1

Why Host the Agent

Copilot CLI's loop, your decisions around it

The mental model, four workflows worth building, and the hook-or-host choice

PART 2

Build It From Zero

Five runnable steps, then a real onboarding tool

Session, events, persona, tools, permissions, then the fresh-eyes friction report

PART 3

Decide What It Sees

One visibility policy at two SDK checkpoints

Write-content checks, command allowlists, and a blind test writer with a referee

PART 4

Prove and Own It

Evidence, owners, and the controls around you

A joined event log, organization policy, and a pre-flight checklist

PART 1

Why Host the Agent

The SDK hosts the same loop as Copilot CLI; your code sets task, persona, tools, and per-call decisions.

🗺️

The mental model

Your app on one side, the Copilot runtime on the other

🎯

What is worth building

An agent with a deliberate perspective

🔀

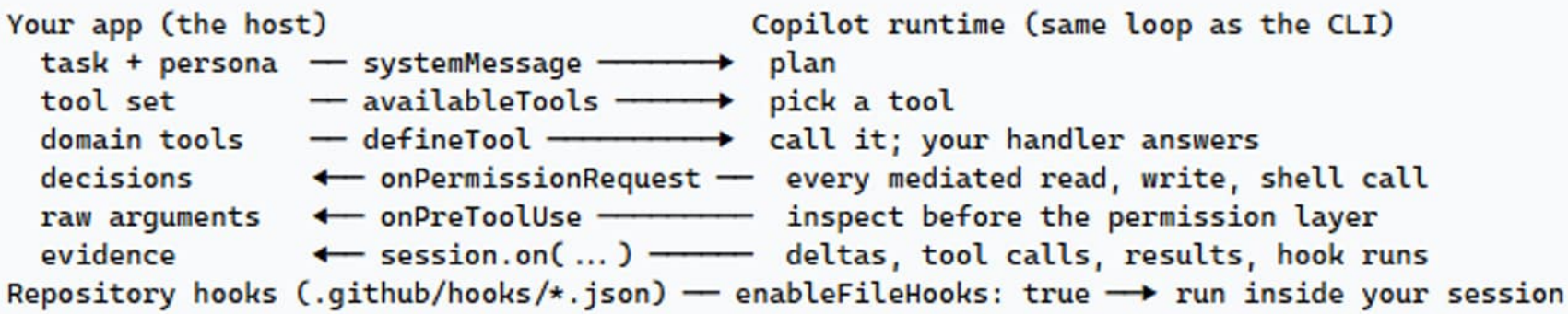
Hook or host

Two ways into the loop, and how they compose

```
Observed precedence probe: repository preToolUse hook vs SDK onPreToolUse
↳ repo deny held while the SDK hook returned allow
```

Your app hosts the loop; the Copilot runtime plans and calls tools

host ↔ runtime (Documented: github/copilot-sdk)



Same agent loop

SDK → JSON-RPC → Copilot runtime, started and stopped for you

Your decisions

Persona, tool set, domain tools, and an answer for every mediated call

Your evidence

Stream events to render, count, budget, and store

Repository floor

Opt in and the team's .github/hooks rules run in your session

📖 Documented: the SDK talks to the Copilot runtime over JSON-RPC; organization policy still sets access, models, and billing.

The payoff is a Copilot agent with a deliberate perspective



Hide the source → onboarding check

Can a newcomer run it from the docs alone? Built: 3 findings, 0 source reads (Observed 2026-10-26)



Hide the implementation → blind tests

Does the API match its contract? Built: host rerun 10 passing, 2 failing (Observed 2026-10-26)



Show only the diff → PR reviewer

Is this PR understandable from its description and diff alone? Pattern to build next



Guide commands only → migration check

Would this migration guide work on a clean

🔑 Same host, three answers per workflow: what the agent may see, what it may use, and who reads its report.

Hooks reach every harness; a host owns its sessions

🪝 Repository hook (preToolUse)

Lives in .github/hooks/ or machine policy

Every harness that loads it: CLI, VS Code, cloud agent

Crash or non-zero exit → deny; timeout → allow

One JSON object on stdout; the reason is shown to the agent

```
{ "permissionDecision": "deny",
  "permissionDecisionReason":
    "Protected path (database): …" }
```

🏠 SDK host (your app)

Lives in your code; sees the whole session

History, state, your app's identity and data

Sets persona, tools, decisions, budgets

You own auth, version pins, lifecycle, cleanup

```
Observed 2026-10-26 · precedence probe
repo preToolUse hook → deny
SDK onPreToolUse     → allow
result → call denied; SDK callbacks
         never invoked
```

🧱 Set enableFileHooks: true and the data team's database deny holds under any host that loads it.

PART 2

Build It From Zero

Five runnable TypeScript steps from npm install to a permission handler, then a real onboarding tool.



Session and events

One answer, then watch every tool call stream



Persona and tools

Decide who the agent is and what it can use



Permissions and payoff

Per-call decisions, then the fresh-eyes check

```
npx tsx sdk-quickstart/05-permissions.ts
↳ shell rejected → 6 approved reads → correct counts
```

Five lines start a session; listeners stream every agent move

sdk-quickstart/01 + 02 (trimmed)

```
const client = new CopilotClient();
const session = await client.createSession({
  model: "auto", streaming: true,
  onPermissionRequest: approveAll });

session.on("assistant.message_delta",
  (e) ⇒ process.stdout.write(e.data.deltaContent));
session.on("tool.execution_start",
  (e) ⇒ console.log("[tool]", e.data.toolName));
session.on("session.idle", () ⇒ console.log("[idle]"));

await session.sendAndWait({
  prompt: "How do I run this project? Two sentences." });
```

👀 Observed 2026-10-26 · trace

[tool] powershell → [tool] view ×3 → streamed answer → [idle]

✅ Observed 2026-10-26 · answer

Copilot agent returned a correct two-sentence answer about examples/

📊 Your app owns the trace

Render it, count tool calls, enforce a budget, store it

🧰 Before you start: Node 20+ · npm install in examples/ · default auth = signed-in Copilot CLI user

A persona works when it states a behavior you can check

sdk-quickstart/03 (trimmed)

```
systemMessage: {
  mode: "customize",
  sections: {
    custom_instructions: {
      action: "append",
      content:
        "You are a first-day engineer who trusts " +
        "only written documentation. " +
        "Cite the file behind every step. …",
    },
  },
},
```

Observed 2026-10-26 · checkable line

Same question; the Copilot agent cited a file on every step

Observed · attitude-only line

An earlier persona describing a mood produced no visible change

customize keeps the guardrails

Edits one named section; replace swaps the whole prompt (Documented)

✍️ Write the persona as a behavior a reviewer can verify: cite, label, report, stop.

Narrow the tool set, add your own; the agent plans with both

sdk-quickstart/04 (trimmed)

```
const lookupOwner = defineTool("lookup_owner", {
  description:
    "Return the team that owns a repository path.",
  parameters: { type: "object",
    properties: { path: { type: "string" } } },
  handler: async ({ path }) ⇒
    /* answer from your own systems */,
});
const session = await client.createSession({
  model: "auto", onPermissionRequest: approveAll,
  availableTools: ["view", "glob", "grep", "lookup_owner"],
  tools: [lookupOwner],
});
```

Practitioner asks

Who reviews node/backend/src/routes/characters.js? Then run git log.

Observed 2026-10-26 · Copilot agent

Called lookup_owner and answered “api-team”

Observed · no shell offered

“couldn't run git log because this session doesn't provide a command-execution tool”

A tool that is never offered is never requested: the tool set shapes the plan before any permission check runs.

The host rejects the shell; the agent recovers through reads

sdk-quickstart/05-permissions.ts (trimmed)

```
const onPermissionRequest: PermissionHandler =
  (request) => {
    if (request.kind === "read")
      return { kind: "approve-once" };
    return {
      kind: "reject",
      feedback: "This session is read-only. " +
        "Use the view tool to inspect files.",
    };
  };
```

Practitioner asks · host rejects

"Count lines in each .ts file": shell call rejected with feedback

Observed 2026-10-26 · recovery

Copilot agent switched to view: 6 approved reads, correct counts

Reads ask too

In-repo reads raise kind: read with request.path, so the host governs what it sees

The feedback string shapes the recovery: the agent read "use the view tool" and did exactly that.

A docs-only agent found that FanHub's two Node.js guides disagree

Host decisions: clean clone per run · docs-only reads, src/ hidden · report_friction tool · 80-call budget

confusing	README.md: Docker Desktop, port 3001 · SETUP.md: no Docker, port 5265	`README vs SETUP`
blocker	“Node.js 18+” only; on Node 24, better-sqlite3 needs Python to build	`node/SETUP.md`
confusing	No native-install fallback; agent used npm exec --package=node@20	`node/SETUP.md`
host deny	Node.js archive download outside the allowlist; filed as friction	`host policy · 1 of 21`

Observed 2026-10-26: 24 tool calls · zero source reads · GET /api/characters → 200 · report goes to the docs owner

PART 3

Decide What It Sees

One visibility policy at two SDK checkpoints; the blind test writer is the deeper verification case.

🔭

Two checkpoints

Raw arguments first, parsed requests second

🔬

What a host can check

Write contents and per-segment commands

⚖️

Blind tests + referee

Host reruns the suite; a referee maps failures

```
Host rerun of the blind-test suite (Observed 2026-10-26)
↳ 10 passing · 2 failing · 1 undocumented contract mismatch
```

One visibility policy, applied at both SDK checkpoints

host-kit/visibility.ts (usage, trimmed)

```
const policy = createVisibilityPolicy({
  repoRoot: checkout.dir,
  hidden: isHiddenFromBlindTester, // never visible
  readable: isBlindReadable,       // read allowlist
  writable: isBlindWritable,       // where it may write
  shellCommands: BLIND_SHELL,      // executables
  customTools: new Set(["api_contract"]),
});

createSession({ … ,
  onPermissionRequest: policy.onPermissionRequest,
  hooks: { onPreToolUse: policy.onPreToolUse } });
```

1️⃣ hooks.onPreToolUse · raw arguments

Catches a grep whose scope sweeps a hidden folder, or a view of a hidden path

2️⃣ onPermissionRequest · parsed request

Checks reads, writes, write contents, and shell executables

🧪 Plain predicates over repo paths

Tested: every rule unit-tests with real request shapes

🔴 Observed 2026-10-26 · host policy, pre-tool: view node/backend/__tests__/readme.md → deny (rule: hidden)

A host checks what a call would do, using the runtime's analysis

📝 Write-content inspection

Copilot agent proposes a test file

Its contents would read hidden source when the test runs

Host policy rejects the write

Tested

```
write  node/backend/__tests__/blind/x.test.js
  contains readFileSync('../../src/…')
host → reject: “That file would read hidden
        material when it runs.”
```

🧩 Per-segment command allowlist

Host splits every shell command

Separators inside quoted arguments stay intact

Each executable checked

Tested with 5 command shapes

```
cd node/backend; npm start | tee log
  → segments: cd · npm · tee   each checked
Observed 2026-10-26: git status --short
  → deny: command-not-allowed:git
```

🏠 A hook script receives the call's arguments; the host also holds the runtime's analysis, the session's state, and its own tools.

The blind tester sees a contract and a running server, and nothing else

Does the API behave the way its contract says, judged by an agent that has never seen the code?

Environment	Clean clone, npm install; the host starts the server on port 5299	`host`
API view	api_contract tool built from SETUP.md, every bug hint stripped	`defineTool`
Hidden	Backend src, frontend, docs/, the database, all Markdown incl. BUGS.md	`visibility policy`
Writable	node/backend/__tests__/blind/ only; failing tests never weakened	`visibility policy`
Proof	The host reruns the suite itself and records TAP output	`host rerun`
Referee	Read-only session reads BUGS.md + source, classifies each failure	`referee session`

Observed 2026-10-26: 18 tool calls · 19 allowed and 2 denied by host policy · 5 test files written

Host rerun: 10 pass, 2 fail; one gap is undocumented

Observed 2026-10-26 · host rerun (TAP) + referee session

```
POST /api/characters returns 201 for valid input
  test: FAIL — received 500
  referee: BUGS.md #19, Missing Input Validation
GET /api/episodes?seasonId=1 returns only season 1
  test: FAIL — episodes from every season
  referee: undocumented · SETUP.md documents seasonId; the route reads season_id
```

🧱 Repository floor active

Observed: 19 preToolUse runs in fresh-eyes, 12 in blind tests

🎲 Runs vary

Run A: 9 passing, 5 failing → #9, #6, #17. The host rerun and referee keep it honest

📦 Executed tests run free

node --test has full process access; a container sets the outer limit

👤 API owner decides

A failing blind test is a disagreement; the API owner picks which side changes

PART 4

Prove and Own It

A joined evidence log, named owners, the organization controls around your app, and a pre-flight checklist.

📄

Evidence and owners

Every decision logged and joined by sessionId

🏢

Controls around you

Policy hooks, access, models, billing, reach

✅

Pre-flight

What to settle before someone else runs it

```
events.jsonl from the blind run (Observed 2026-10-26)
↳ repo-hook + host-policy + runtime + host, joined by sessionId
```

Every run leaves a joined event log and a named findings owner

Observed 2026-10-26 · .runs/blind-tests/events.jsonl (4 lines, trimmed)


```
{"layer":"repo-hook","event":"hook.end","sessionId":"909bf25c-…","hook":"preToolUse","success":true}
{"source":"host-policy","layer":"pre-tool","kind":"view","decision":"deny","rule":"hidden"}
{"layer":"runtime","event":"tool.complete","sessionId":"909bf25c-…","success":true}
{"layer":"host","event":"session-end","sessionId":"909bf25c-…","toolCalls":18,"allow":19,"deny":2}
```


⚖️ **Decisions apart from executions**

An allowed call can still fail

🔒 **Names, rules, counts**

File contents and command output stay out by construction

🏁 **Host is the source of truth**

It reruns the suite; the friction report comes from tool calls

👥 Platform team publishes policy → practitioner defines the workflow → Copilot agent works → host records and verifies → docs or API owner accepts or routes

Your app runs inside decisions other people own

Documented: GitHub Copilot hooks reference, Copilot policies, supported models, SDK authentication

Policy hooks	Load machine-wide before repo hooks; run even under disableAllHooks	platform team
Access	Who may use Copilot where; verify the identity your app runs as	org admins
Models	Admins set availability; model: auto is a request; log what ran	admins · your app
Billing	User or token route: Copilot policy and billing. BYOK: provider terms	auth route
Tools and MCP	availableTools plus managed settings; managedApprovalRequired → human	admins · handler
Process reach	OS account, container, network: the outer boundary for installs	platform · service owners

Policy hooks follow the same contract: a timeout is fail-open, so keep them local and fast

Settle five things before someone else runs your app

📋 Pre-flight checklist

- **Pin the SDK; log the runtime version**
- **Clean, pinned checkout for every run**
- **Budget, timeout, and cleanup in finally**
- **Test each rule; enableFileHooks: true**
- **Name auth route, model, report owner**

🪜 How deep to go

Start at the shallowest layer that works

Each step down adds code you own

```
Steering note at session start enough?
  → sessionStart hook. Done.
A call must never run, in any harness?
  → preToolUse or policy hook, fast
Needs persona, tools, visibility,
budget, or a report?
  → SDK host + enableFileHooks: true
```

🧪 Tested: npm test runs 19 tests across the repository hook decisions and the visibility rules.

THE VALUE RECAP

From a general agent to a host with a deliberate view

Before

- The agent sees the whole repository and answers “make this work”
- Tool set and permissions follow the harness defaults
- Findings live in the agent's closing summary
- Repository rules apply inside the editor and CLI

After

- A docs-only agent: 24 tool calls, zero source reads, three findings
- The host rejects a shell call with feedback; the agent recovers via 6 reads
- The host reruns 12 blind tests; a referee maps #19 and one undocumented gap
- The repository deny holds under an SDK allow, inside every host

0
source files opened by the fresh-eyes agent (Observed)

10 / 2
passing / failing on the blind-test host rerun (Observed)

19
hook and visibility-rule tests in examples/ (Tested)

✅ WHAT YOU CAN DO TODAY

Today

- cd tech-talks/harness-fundamentals/examples and run npm install
- Run npx tsx sdk-quickstart/01-hello.ts for your first session
- Run 05-permissions.ts and look for a [reject] shell line, then approved reads

This Week

- Run npm test and confirm all 19 hook and visibility tests pass
- Run npm run fresh-eyes and open friction-report.md
- Run npm run blind-tests; find host denials and repo preToolUse runs for one sessionId

This Month

- Copy host-kit/ and one app into a workflow of your own
- Rewrite the rules for what the agent sees, writes, and runs; test each with real paths
- Run it on a clean clone and hand the first report to its owner

🔑 Key Takeaway

Pick one workflow that needs a fresh, constrained perspective, and decide what your agent sees, uses, and who reads its report.

REFERENCES

Copilot SDK

GitHub Copilot SDK repository
Architecture, sessions, tools, hooks, examples

@github/copilot-sdk on npm
1.0.26 used in this talk: permission kinds, systemMessage modes, enableFileHooks

Copilot SDK authentication
Signed-in user, token, and BYOK routes

Hooks and Policy

GitHub Copilot hooks reference
preToolUse decisions, fail-closed and fail-open rules, policy hook locations

VS Code agent hooks (Preview)
The editor's hook format and event coverage

Copilot policies for enterprises and organizations
AI controls for features, agents, and models

Supported AI models in GitHub Copilot
Availability by plan and surface

Related Content

tech-talks/harness-fundamentals/examples
Quickstart steps 01–05, host-kit, fresh-eyes and blind-test apps, 19 tests

Copilot Primitives · Custom Agents
The building blocks a host configures



Harness Fundamentals

GETTING INSIDE THE AGENT LOOP

24 · 0 · 3

Tool calls, source reads, and onboarding findings from a docs-only agent

10 ✓ · 2 ✗

Blind-test host rerun; the referee mapped #19 and labeled one gap undocumented

deny holds

A repository preToolUse deny held while the SDK hook returned allow

Which workflow on your team would benefit from an agent that sees less?